

به نام خداوند جان و خرد

آزمایش چهارم آزمایشگاه طراحی سیستم‌های دیجیتال

اعضای گروه

پوریا رحمانی ۴۰۲۱۱۴۱۸

امیرمحمد شربتی ۴۰۲۱۰۶۱۱۲

موضوع آزمایش

طراحی یک پشته با پهنای ۴ بیت و عمق ۸ با ورودی‌های یک بیتی RstN (ریست active low)، Clk (کلاک)، Pop، Push و ورودی ۴ بیتی Data_In، خروجی‌های یک بیتی Full (یک است اگر و فقط اگر پشته پر باشد)، Empty (صفر است اگر و فقط اگر پشته خالی باشد) و خروجی ۴ بیتی Data_out.

پیاده‌سازی با verilog

برای خود پشته از آرایه‌ای از vector ها استفاده کردیم (reg[3:0]stack_instance[7:0]) و برای اشاره گر پشته از یک vector چهار بیتی (reg[3:0]pointer) کمک گرفتیم که همیشه به اولین خانه خالی پشته اشاره می‌کند (و اگر پشته پر باشد برابر ۸ است) منطق پیاده‌سازی به این شکل است که با هر لبه بالارونده clock یا با لبه پایین رونده Rstn (وقتی Rstn را به صفر تغییر می‌دهیم):

- اگر Rstn صفر بود، پشته را خالی می‌کنیم. یعنی Full را برابر صفر می‌گذاریم، Empty را برابر صفر می‌گذاریم (که یعنی پشته خالی است)، pointer را هم برابر صفر می‌گذاریم (که باز مشخص می‌کند چیزی در پشته نیست) و Data_out را نیز برابر صفر قرار می‌دهیم.

```
if(!RstN) begin
    Empty <= 1'b0;
    Full <= 1'b0;
    pointer <= 4'b0;
    Data_out <= 4'b0;
```

- اگر Rstn یک بود، یعنی لبه بالارونده clock رخ داده است. بنابراین باز شرایط را به دو حالت تقسیم می‌کنیم:
 - اگر سیگنال Push یک بود (اگر هر دوی Push و Pop یک بودند، Push را به Pop ترجیح می‌دهیم) و پشته پر نبود (سیگنال Full یک نبود)، سیگنال Empty را یک می‌کنیم (چون بعد یک push قطعاً پشته خالی نخواهد بود) و در جایگاه Data_In، pointer را می‌نویسیم و pointer را یکی زیاد می‌کنیم. بعلاوه اگر مقدار جدید pointer برابر ۸ شده بود، سیگنال Full را یک می‌کنیم. تنها نکته این است که از آنجا که در توصیف رفتاری باید از non-blocking assignment استفاده کنیم و با این نوع assignment، تا آخر آن واحد زمانی، تخصیص‌ها رخ نمی‌دهد، باید در شرط خود ۷ بودن pointer را بررسی کنیم. (چون اگر در این واحد زمانی ۷ باشد، در ابتدای واحد زمانی بعدی ۸ خواهد بود).

```

end else if(Push && !Full) begin
    stack_instance[pointer] <= Data_In;
    pointer <= pointer + 1;
    Full <= (pointer == 4'd7);
    Empty <= 1'b1;
end else if(Pop && Empty) begin

```

○ اگر سیگنال Pop یک بود ولی سیگنال Push یک نبود و Empty هم یک بود (پشته خالی نبود) باید در Data_out مقدار stack در جایگاه $pointer - 1$ را بریزیم، pointer را یکی کم کنیم، Full را صفر کنیم (چون بعد از یک عملیات Pop قطعا پشته پر نخواهد بود) و اگر pointer صفر بشود، Empty را هم یک می‌کنیم. (پشته خالی شده‌است). باز هم توجه کنید به علت استفاده از non-blocking assignment، pointer تا انتهای آن واحد زمانی صفر نخواهد شد و در نتیجه در شرط، باید یک بودن pointer را چک کنیم.

```

end else if(Pop && Empty) begin
    pointer <= pointer - 1;
    Data_out <= stack_instance[pointer-1];
    Empty <= !(pointer == 4'd1);
    Full <= 1'b0;
end

```

تست بنچ

برای انجام راحت‌تر تست‌ها، یک task برای push و یک task برای pop نوشتیم.

task مربوط به push به این صورت است که ابتدا برای اطمینان تا لبه بالا رونده clock صبر می‌کنیم، سپس push را برابر یک و pop را برابر صفر قرار می‌دهیم. Data_In را هم برابر ورودی task قرار می‌دهیم و باز هم تا لبه بالارونده clock صبر می‌کنیم. بعد push را صفر می‌کنیم:

```

task make_a_push(input [3:0] value_to_push);
begin
    @(posedge Clk);
    Data_In = value_to_push;
    Push = 1; Pop = 0;
    @(posedge Clk);
    Push = 0;
end
endtask

```

task مربوط به Pop به این صورت است که ابتدا تا لبه بالا رونده Clock صبر می‌کنیم. سپس Push را برابر صفر و Pop را برابر یک قرار می‌دهیم، تا لبه بالا رونده clock صبر می‌کنیم و در نهایت Pop را دوباره صفر می‌کنیم. (شکل در صفحه بعد)

```
task make_a_pop();
begin
    @(posedge Clk);
    Push = 0; Pop = 1;
    @(posedge Clk);
    Pop = 0;
end
endtask
```

حالا سناریوهای زیر را تست می‌کنیم. در ابتدا RstN را صفر می‌گذاریم تا مدار reset شود و سیگنال‌های Full ، Empty و Data_out را چک می‌کنیم که درست (برابر صفر) باشند:

```
RstN = 1'b0;
@(posedge Clk);
#1;
$display("Data_out = %0d", Data_out);
$display("Full = %0d", Full);
$display("Empty = %0d", Empty);
if(Empty == 1'b0 && Full == 1'b0 && Data_out == 0) $display("reset was Sucessful!");
else $display("reset Failed!");
$display("-----");
```

(توجه کنید یک واحد تاخیر قبل از چاپ کردن‌ها برای اطمینان از این موضوع است که \$display بعد از assign شدن مقادیر جدید، نتیجه را چاپ خواهد کرد و دقیقا در لبه clock که تغییر مقادیر اتفاق می‌افتد نیست تا race بوجود آید)

که نتیجه به این صورت بود:

```
# Data_out = 0
# Full = 0
# Empty = 0
# reset was Sucessful!
# -----
```

سپس، درحالی‌که RstN همچنان صفر بود، عدد یک را Push می‌کنیم. سپس RstN را یک کرده و Pop می‌کنیم و بررسی می‌کنیم که Data_out تغییر نکرده باشد. (دلیل یک واحد تاخیر هم مشابه بخش قبل است)

```
//-----
$display("Pushing while RstN = 0 ...");
Push = 1;
Data_In = 4'd7;
@(posedge Clk);
//-----
RstN = 1'b1;
Pop = 1'b1;
Push = 1'b0;
@(posedge Clk);
#1;
$write("Reset was 0 while pushing and Data_out = %0d, so your stack ", Data_out);
if(Data_out == 0) $display("passed this test!");
else $display("failed this test");
$display("-----");
```

```
# -----
# Pushing while RstN = 0 ...
# Reset was 0 while pushing and Data_out = 0, so your stack passed this test!
# -----
```

سپس عدد ۷ را push و سپس pop می‌کنیم تا بررسی کنیم آیا Data_out به ۷ تغییر کرده‌است یا خیر:

```
//-----
$display("Pushing number 7 into stack...");
make_a_push(4'd7);
$display("now Popping it...");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
if(Data_out == 4'd7) $display("Passed the single push -> pop task!");
else $display("Failed the single push -> pop task!");
$display("-----");

# -----
# Pushing number 7 into stack...
# now Popping it...
# Data_out = 7
# Passed the single push -> pop task!
# -----
```

سپس هشت بار در پشته push کردیم (اعداد فرد به ترتیب از ۱ تا ۱۵) و هربار Full را چاپ کردیم تا ببینیم صرفاً آخرین بار Full برابر یک می‌شود. سپس یک push اضافه هم کردیم (عدد ۸) تا بعد با Pop کردن ببینیم ۸ به علت پر بودن وارد پشته نشده و سر پشته همچنان ۱۵ است. سپس، ۸ بار Pop کردیم و هربار Data_out و Empty را چاپ کردیم تا ببینیم اعداد به ترتیب درستی وارد شده‌اند و تنها در آخرین بار Empty برابر صفر شده‌است. در نهایت نیز یک Pop اضافه انجام دادیم و سپس باز Data_out را چاپ کردیم تا بررسی کنیم Pop در حالت خالی بودن، تاثیری روی Data_out نمی‌گذارد و Data_out برابر یک باقی می‌ماند. یک integer هم تعریف کردیم تا اگر حداقل یکی از نتایج مورد انتظار نبود، آن را set کنیم و با توجه به آن موفقیت آمیز بودن یا نبودن تست را چاپ کردیم. (بلوک‌های کد و نتیجه را از چپ به راست بخوانید!)

```
$display("pushing 8 values to the stack....");
$display("pushing 1...");
make_a_push(4'd1);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;

$display("pushing 3...");
make_a_push(4'd3);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;

$display("pushing 5...");
make_a_push(4'd5);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;

$display("pushing 7...");
make_a_push(4'd7);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;

$display("pushing 9...");
make_a_push(4'd9);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;
```

```
$display("pushing 11...");
make_a_push(4'd11);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;

$display("pushing 13...");
make_a_push(4'd13);
#1;
$display("Full = %0d\n", Full);
if(Full == 1'b1) failed = 1;
if(failed) $display("You made the stack full too early!");

$display("pushing 15...");
make_a_push(4'd15);
#1;
$display("now you should make the Full signal true!");
$display("Full = %0d\n", Full);
if(Full == 1) $display("You made that successfully!");
else $display("Failed!");
$display("-----");
//
$display("pushing 8 as an extra number!(should not affect the stack)");
make_a_push(4'd8);
$display("-----");
//-----
```

```
failed = 0;
$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d\n", Empty);
if(Empty == 1'b0 || Data_out != 4'd15) failed = 1;

$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d\n", Empty);
if(Empty == 1'b0 || Data_out != 4'd13) failed = 1;

$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d\n", Empty);
if(Empty == 1'b0 || Data_out != 4'd11) failed = 1;

$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d\n", Empty);
if(Empty == 1'b0 || Data_out != 4'd9) failed = 1;

$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d\n", Empty);
if(Empty == 1'b0 || Data_out != 4'd7) failed = 1;
```

```

$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d", Empty);
if(Empty == 1'b0 || Data_out != 4'd5) failed = 1;

$display("Popping ....");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d", Empty);
if(Empty == 1'b0 || Data_out != 4'd5) failed = 1;

$display("Popping...");
$display("now you should make Empty zero");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
$display("Empty = %0d", Empty);
if(Empty == 1'b1 || Data_out != 4'd1) failed = 1;

if(!failed) $display("Passed the 8 push -> pop test!");
else $display("Failed the 8 push -> pop test!");
$display("-----");
$display("popping one time more!");
make_a_pop();
#1;
$display("Data_out = %0d", Data_out);
if(Data_out == 4'd1) $display("Passed the extra pop didn't affect!");
else $display("Failed the extra pop affected the stack!");

```

```

# -----
# Pushing 8 values to the stack...
# Pushing 1...
# Full = 0
#
# Pushing 3...
# Full = 0
#
# Pushing 5...
# Full = 0
#
# Pushing 7...
# Full = 0
#
# Pushing 9...
# Full = 0
#
# Pushing 11...
# Full = 0
#
# Pushing 13...
# Full = 0
#
# Pushing 15...
# now you should make the Full signal true!
# Full = 1
#
# You made that successfully!
# -----
# Pushing 8 as an extra number!(should not affect the stack)
# -----

Popping ....
Data_out = 15
Empty = 1

Popping ....
Data_out = 13
Empty = 1

Popping ....
Data_out = 11
Empty = 1

Popping ....
Data_out = 9
Empty = 1

Popping ....
Data_out = 7
Empty = 1

Popping ....
Data_out = 5
Empty = 1

Popping ....
Data_out = 3
Empty = 1

Popping...
now you should make Empty zero
Data_out = 1
Empty = 0

Passed the 8 push -> pop test!
-----
popping one time more!
Data out = 1
Passed! the extra Pop didn't affect!
-----

```

در نهایت نیز تستی با چند Push و Pop تصادفی انجام دادیم. به این صورت که ابتدا چهار بار Push سپس دوبار Pop ، یک بار دیگر Push دوبار دیگر Pop ، یکبار Push و دوبار Pop کردیم و در هر مرحله از Pop ، Data_out را چاپ کردیم و در نهایت نیز Empty را چاپ کردیم و مطمئن شدیم پشته به درستی خالی شده است.(باز هم برای اعلام Pass یا Fail شدن از مکانیزم مشابه بخش پیش استفاده شده است)

```

$display("-----");
failed = 0;
$display("Random Push and pop test...\n");
$display("Pushing 8...");
make_a_push(4'd8);

$display("Pushing 10...");
make_a_push(10);

$display("Pushing 12...");
make_a_push(12);

$display("Pushing 5...");
make_a_push(5);

$display("Popping...");
make_a_pop();
#1;
$display("Data_out = %0d\n", Data_out);
if(Data_out != 4'd5) failed = 1;

$display("Popping...");
make_a_pop();
#1;
$display("Data_out = %0d\n", Data_out);
if(Data_out != 4'd12) failed = 1;

$display("Pushing 14...");
make_a_push(14);

```

```

$display("Popping...");
make_a_pop();
#1;
$display("Data_out = %0d\n", Data_out);
if(Data_out != 4'd14) failed = 1;

$display("Popping...");
make_a_pop();
#1;
$display("Data_out = %0d\n", Data_out);
if(Data_out != 4'd10) failed = 1;

$display("Pushing 13...");
make_a_push(13);

$display("Popping...");
make_a_pop();
#1;
$display("Data_out = %0d\n", Data_out);
if(Data_out != 4'd13) failed = 1;

$display("Popping...");
make_a_pop();
#1;
$display("Data_out = %0d\n", Data_out);
if(Data_out != 4'd8) failed = 1;

$display("Empty = %0d", Empty);
if(Empty != 1'd0) failed = 1;

if(!failed) $display("random push-pop test was successful!");
else $display("random push-pop test Failed!");

$finish();

```

```

# -----
# Random Push and pop test...
#
# Pushing 8...
# Pushing 10...
# Pushing 12...
# Pushing 5...
# Popping...
# Data_out = 5
#
# Popping...
# Data_out = 12
#
# Pushing 14...
# Popping...
# Data_out = 14
#
# Popping...
# Data_out = 10
#
# Pushing 13...
# Popping...
# Data_out = 13
#
# Popping...
# Data_out = 8
#
# Empty = 0
# random push-pop test was successful!

```

یک فایل dump.vcd به ضمیمه وجود دارد که شکل موج این testbench در آن هست.